After pre-training, the GPT model can be fine-tuned for a specific task by providing it with a smaller, task-specific dataset. Fine-tuning involves adjusting the parameters of the model to better fit the task at hand. For example, the model can be fine-tuned for tasks such as classification, similarity scoring, or question answering.

The GPT architecture has since been improved and extended by OpenAI with the release of subsequent models such as GPT-2, GPT-3, GPT-3.5, and GPT-4. These models are trained on larger datasets and have larger capacities, so they can generate more complex and coherent text. The GPT models have been widely adopted by researchers and industry practitioners and have contributed to significant advancements in natural language processing tasks.

In this chapter, we will build our own variation of the original GPT model, trained on less data, but still utilizing the same components and underlying principles.

Running the Code for This Example

The code for this example can be found in the Jupyter notebook located at *notebooks/09_transformer/01_gpt/gpt.ipynb* in the book repository.

The code is adapted from the excellent GPT tutorial created by Apoorv Nandan available on the Keras website.

The Wine Reviews Dataset

We'll be using the Wine Reviews dataset that is available through Kaggle. This is a set of over 130,000 reviews of wines, with accompanying metadata such as description and price.

You can download the dataset by running the Kaggle dataset downloader script in the book repository, as shown in Example 9-1. This will save the wine reviews and accompanying metadata locally to the */data* folder.

Example 9-1. Downloading the Wine Reviews dataset

```
bash scripts/download_kaggle_data.sh zynicide wine-reviews
```

The data preparation steps are identical to the steps used in Chapter 5 for preparing data for input into an LSTM, so we will not repeat them in detail here. The steps, as shown in Figure 9-1, are as follows:

1. Load the data and create a list of text string descriptions of each wine.
2. Pad punctuation with spaces, so that each punctuation mark is treated as a separate word.
3. Pass the strings through a `TextVectorization` layer that tokenizes the data and pads/clips each string to a fixed length.
4. Create a training set where the inputs are the tokenized text strings and the outputs to predict are the same strings shifted by one token.

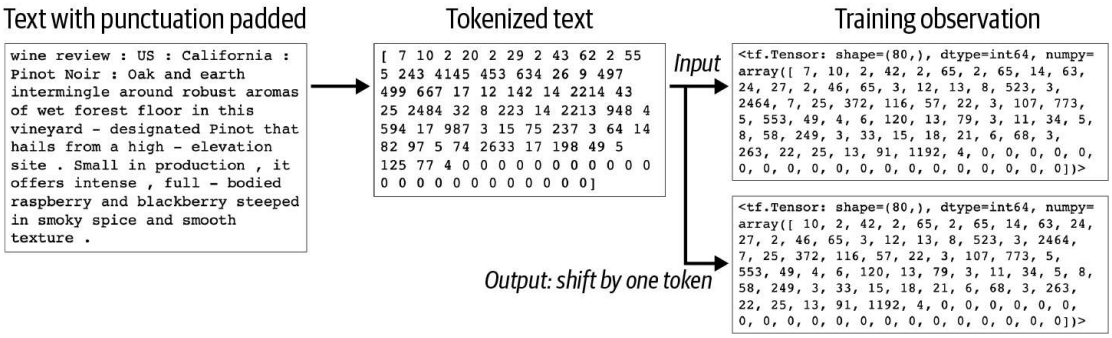

Figure 9-1. Data processing for the Transformer

Attention

The first step to understanding how GPT works is to understand how the *attention mechanism* works. This mechanism is what makes the Transformer architecture unique and distinct from recurrent approaches to language modeling. When we have developed a solid understanding of attention, we will then see how it is used within Transformer architectures such as GPT.

When you write, the choice that you make for the next word in the sentence is influenced by other words that you have already written. For example, suppose you start a sentence as follows:

```
The pink elephant tried to get into the car but it was too
```

Clearly, the next word should be something synonymous with *big*. How do we know this?

Certain other words in the sentence are important for helping us to make our decision. For example, the fact that it is an elephant, rather than a sloth, means that we prefer *big* rather than *slow*. If it were a swimming pool, rather than a car, we might choose *scared* as a possible alternative to *big*. Lastly, the action of *getting into* the car implies that size is the problem—if the elephant was trying to *squash* the car instead, we might choose *fast* as the final word, with *it* now referring to the car.

Other words in the sentence are not important at all. For example, the fact that the elephant is pink has no influence on our choice of final word. Equally, the minor words in the sentence (*the*, *but*, *it*, etc.) give the sentence grammatical form, but here aren't important to determine the required adjective.

In other words, we are *paying attention* to certain words in the sentence and largely ignoring others. Wouldn't it be great if our model could do the same thing?

An attention mechanism (also know as an *attention head*) in a Transformer is designed to do exactly this. It is able to decide where in the input it wants to pull information from, in order to efficiently extract useful information without being clouded by irrelevant details. This makes it highly adaptable to a range of circumstances, as it can decide where it wants to look for information at inference time.

In contrast, a recurrent layer tries to build up a generic hidden state that captures an overall representation of the input at each timestep. A weakness of this approach is that many of the words that have already been incorporated into the hidden vector will not be directly relevant to the immediate task at hand (e.g., predicting the next word), as we have just seen. Attention heads do not suffer from this problem, because they can pick and choose how to combine information from nearby words, depending on the context.

Queries, Keys, and Values

So how does an attention head decide where it wants to look for information? Before we get into the details, let's explore how it works at a high level, using our *pink elephant* example.

Imagine that we want to predict what follows the word *too*. To help with this task, other preceding words chime in with their opinions, but their contributions are weighted by how confident they are in their own expertise in predicting words that follow *too*. For example, the word *elephant* might confidently contribute that it is more likely to be a word related to size or loudness, whereas the word *was* doesn't have much to offer to narrow down the possibilities.

In other words, we can think of an attention head as a kind of information retrieval system, where a *query* ("What word follows *too*?") is made into a *key/value* store (other words in the sentence) and the resulting output is a sum of the values, weighted by the *resonance* between the query and each key.

We will now walk through the process in detail (Figure 9-2), again with reference to our *pink elephant* sentence.

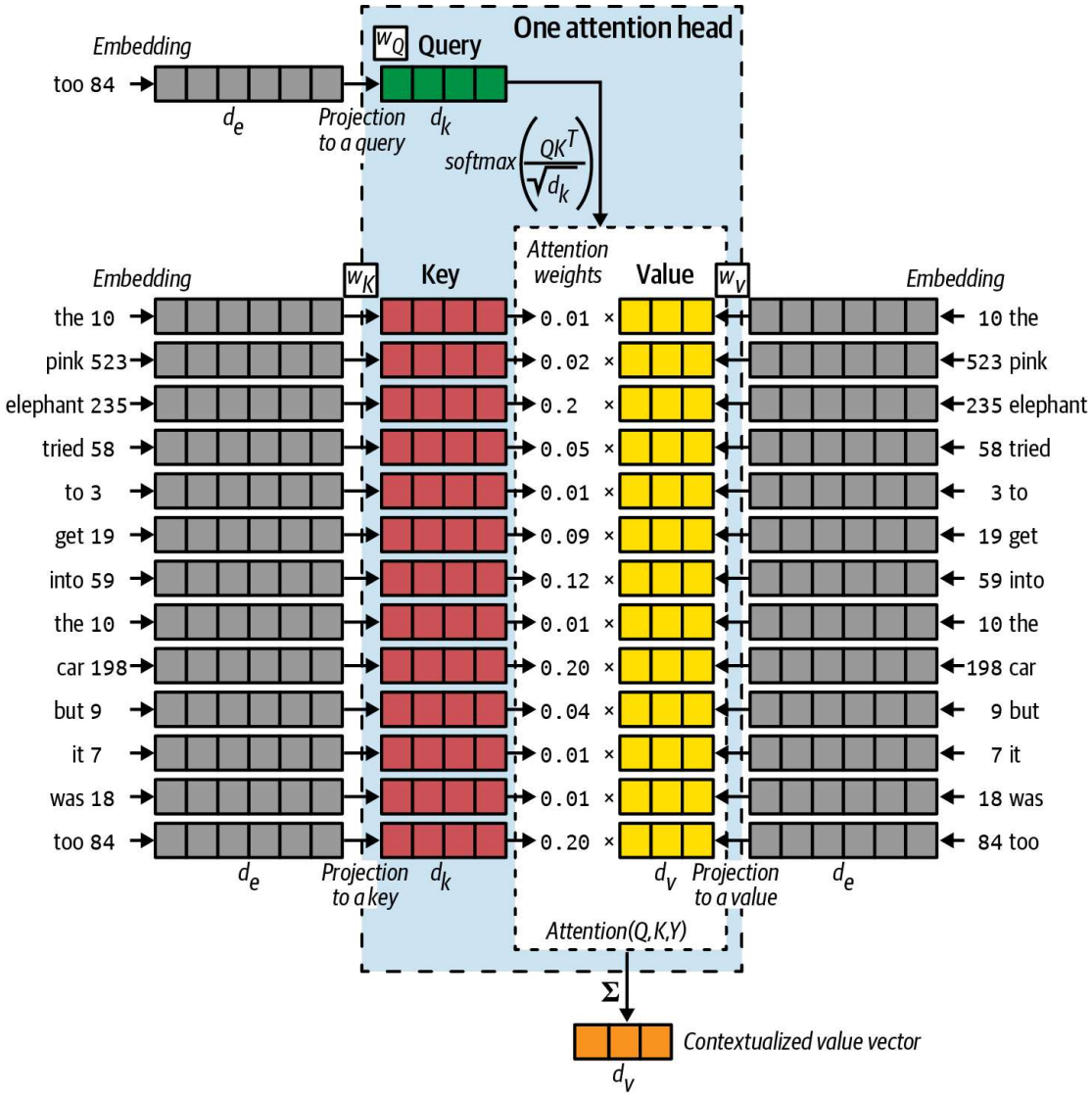

Figure 9-2. The mechanics of an attention head

The *query* (Q) can be thought of as a representation of the current task at hand (e.g., "What word follows *too*?"). In this example, it is derived from the embedding of the word *too*, by passing it through a weights matrix W_Q to change the dimensionality of the vector from d_e to d_k.

The *key* vectors (K) are representations of each word in the sentence—you can think of these as descriptions of the kinds of prediction tasks that each word can help with. They are derived in a similar fashion to the query, by passing each embedding through a weights matrix W_K to change the dimensionality of each vector from d_e to d_k. Notice that the keys and the query are the same length (d_k).

Inside the attention head, each key is compared to the query using a dot product between each pair of vectors (QK^T). This is why the keys and the query have to be the same length. The higher this number is for a particular key/query pair, the more the key resonates with the query, so it is allowed to make more of a contribution to the output of the attention head. The resulting vector is scaled by $\sqrt{d_k}$ to keep the variance of the vector sum stable (approximately equal to 1), and a softmax is applied to ensure the contributions sum to 1. This is a vector of *attention weights*.

The *value* vectors (V) are also representations of the words in the sentence—you can think of these as the unweighted contributions of each word. They are derived by passing each embedding through a weights matrix W_V to change the dimensionality of each vector from d_e to d_v. Notice that the value vectors do not necessarily have to have the same length as the keys and query (but often do, for simplicity).

The value vectors are multiplied by the attention weights to give the *attention* for a given Q, K, and V, as shown in Equation 9-1.

Equation 9-1. Attention equation

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

To obtain the final output vector from the attention head, the attention is summed to give a vector of length d_v. This *context vector* captures a blended opinion from words in the sentence on the task of predicting what word follows *too*.

Multihead Attention

There's no reason to stop at just one attention head! In Keras, we can build a `MultiHeadAttention` layer that concatenates the output from multiple attention heads, allowing each to learn a distinct attention mechanism so that the layer as a whole can learn more complex relationships.

The concatenated outputs are passed through one final weights matrix W_O to project the vector into the desired output dimension, which in our case is the same as the input dimension of the query (d_e), so that the layers can be stacked sequentially on top of each other.

Figure 9-3 shows how the output from a `MultiHeadAttention` layer is constructed. In Keras we can simply write the line shown in Example 9-2 to create such a layer.

Example 9-2. Creating a `MultiHeadAttention` layer in Keras

```
layers.MultiHeadAttention(
    num_heads = 4, ❶
    key_dim = 128, ❷
    value_dim = 64, ❸
    output_shape = 256 ❹
    )
```

❶ This multihead attention layer has four heads.

❷ The keys (and query) are vectors of length 128.

❸ The values (and therefore also the output from each head) are vectors of length 64.

❹ The output vector has length 256.

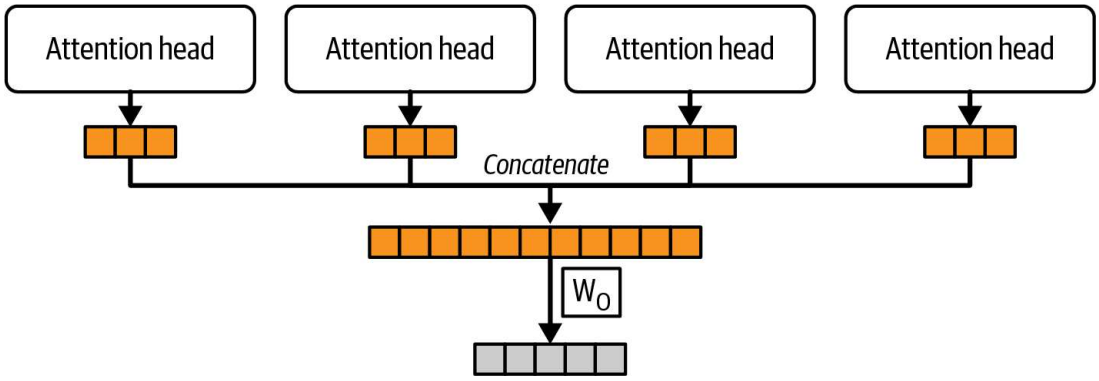

Figure 9-3. A multihead attention layer with four heads

Causal Masking

So far, we have assumed that the query input to our attention head is a single vector. However, for efficiency during training, we would ideally like the attention layer to be able to operate on every word in the input at once, predicting for each what the subsequent word will be. In other words, we want our GPT model to be able to handle a group of query vectors in parallel (i.e., a matrix).

You might think that we can just batch the vectors together into a matrix and let linear algebra handle the rest. This is true, but we need one extra step—we need to apply a mask to the query/key dot product, to avoid information from future words leaking through. This is known as *causal masking* and is shown in Figure 9-4.

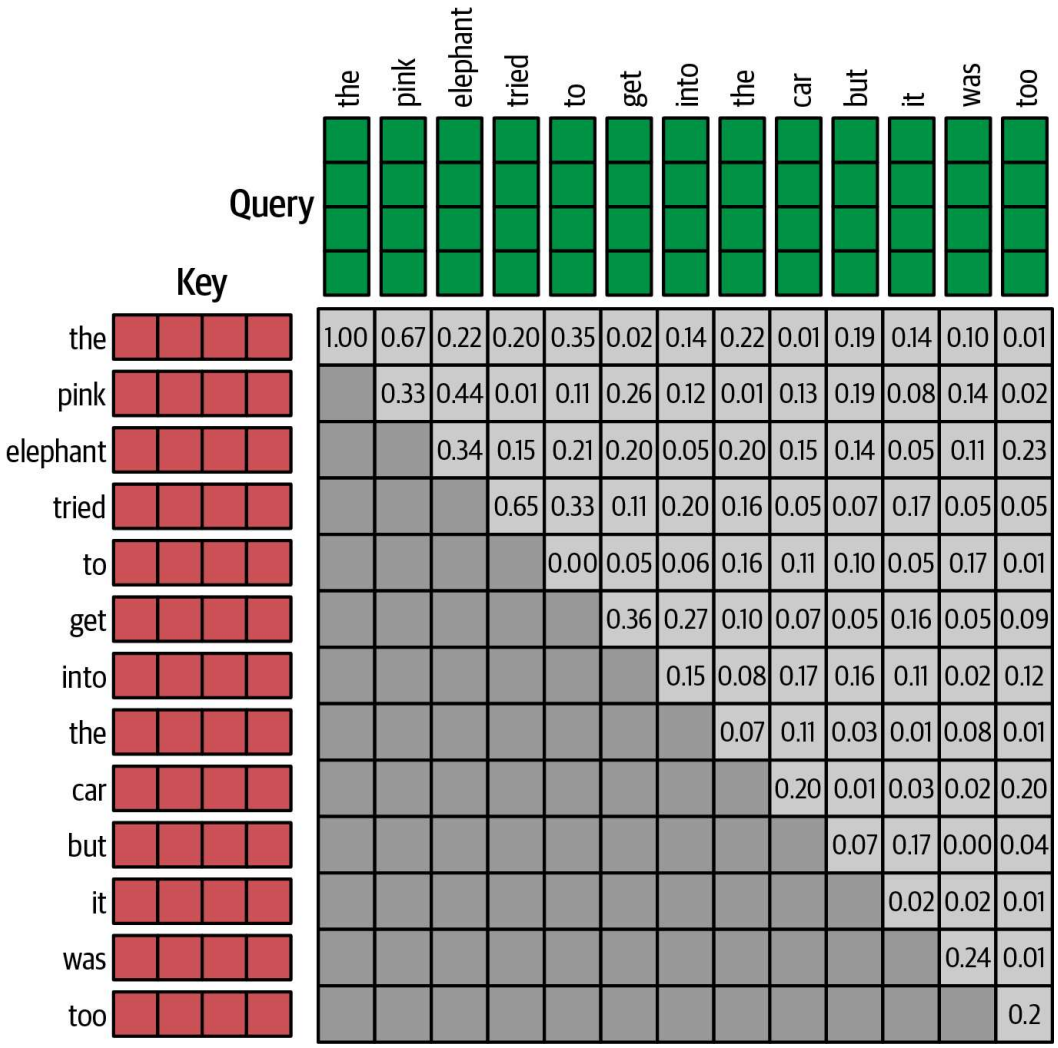

Figure 9-4. Matrix calculation of the attention scores for a batch of input queries, using a causal attention mask to hide keys that are not available to the query (because they come later in the sentence)

Without this mask, our GPT model would be able to perfectly guess the next word in the sentence, because it would be using the key from the word itself as a feature! The code for creating a causal mask is shown in Example 9-3, and the resulting `numpy` array (transposed to match the diagram) is shown in Figure 9-5.

Example 9-3. The causal mask function

```
def causal_attention_mask(batch_size, n_dest, n_src, dtype):
    i = tf.range(n_dest)[:, None]
    j = tf.range(n_src)
    m = i >= j - n_src + n_dest
    mask = tf.cast(m, dtype)
    mask = tf.reshape(mask, [1, n_dest, n_src])
```

```
    mult = tf.concat(
        [tf.expand_dims(batch_size, -1), tf.constant([1, 1], dtype=tf.int32)], 0
    )
    return tf.tile(mask, mult)

np.transpose(causal_attention_mask(1, 10, 10, dtype = tf.int32)[0])
```

```
array([[1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
       [0, 1, 1, 1, 1, 1, 1, 1, 1, 1],
       [0, 0, 1, 1, 1, 1, 1, 1, 1, 1],
       [0, 0, 0, 1, 1, 1, 1, 1, 1, 1],
       [0, 0, 0, 0, 1, 1, 1, 1, 1, 1],
       [0, 0, 0, 0, 0, 1, 1, 1, 1, 1],
       [0, 0, 0, 0, 0, 0, 1, 1, 1, 1],
       [0, 0, 0, 0, 0, 0, 0, 1, 1, 1],
       [0, 0, 0, 0, 0, 0, 0, 0, 1, 1],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 1]], dtype=int32)
```

Figure 9-5. The causal mask as a `numpy` array—1 means unmasked and 0 means masked

Causal masking is only required in *decoder Transformers* such as GPT, where the task is to sequentially generate tokens given previous tokens. Masking out future tokens during training is therefore essential.

Other flavors of Transformer (e.g., *encoder Transformers*) do not need causal masking, because they are not trained to predict the next token. For example Google's BERT predicts masked words within a given sentence, so it can use context from both before and after the word in question.[3]

We will explore the different types of Transformers in more detail at the end of the chapter.

This concludes our explanation of the multihead attention mechanism that is present in all Transformers. It is remarkable that the learnable parameters of such an influential layer consist of nothing more than three densely connected weights matrices for each attention head (W_Q, W_K, W_V) and one further weights matrix to reshape the output (W_O). There are no convolutions or recurrent mechanisms at all in a multihead attention layer!

Next, we shall take a step back and see how the multihead attention layer forms just one part of a larger component known as a *Transformer block*.

The Transformer Block

A *Transformer block* is a single component within a Transformer that applies some skip connections, feed-forward (dense) layers, and normalization around the multi-head attention layer. A diagram of a Transformer block is shown in Figure 9-6.

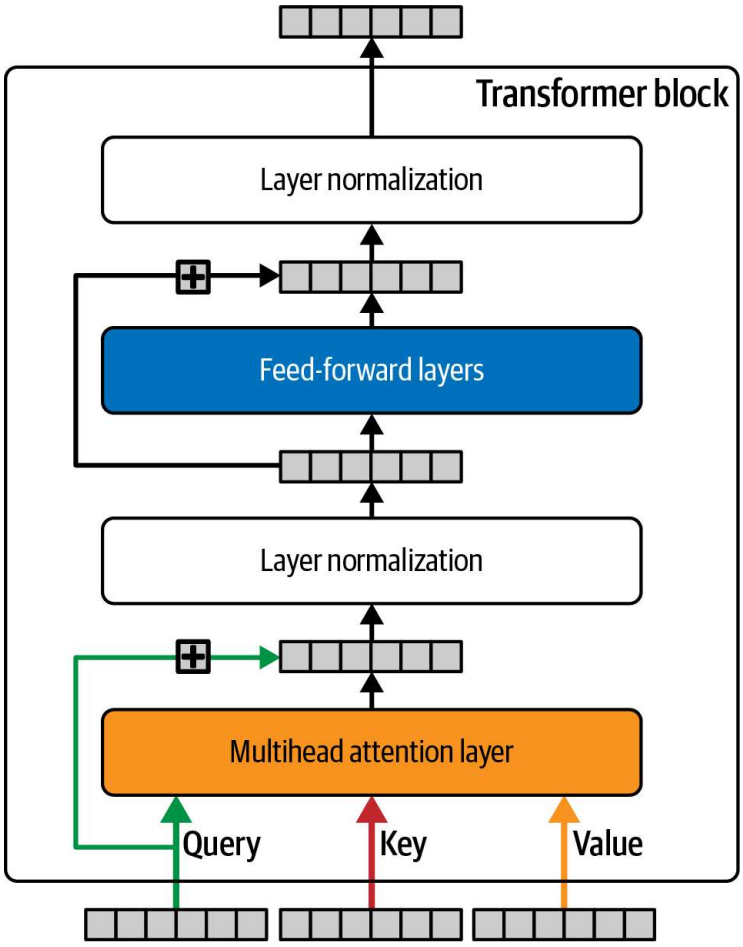

Figure 9-6. A Transformer block

Firstly, notice how the query is passed around the multihead attention layer to be added to the output—this is a skip connection and is common in modern deep learning architectures. It means we can build very deep neural networks that do not suffer as much from the vanishing gradient problem, because the skip connection provides a gradient-free *highway* that allows the network to transfer information forward uninterrupted.

Secondly, *layer normalization* is used in the Transformer block to provide stability to the training process. We have already seen the batch normalization layer in action throughout this book, where the output from each channel is normalized to have a

mean of 0 and standard deviation of 1. The normalization statistics are calculated across the batch and spatial dimensions.

In contrast, layer normalization in a Transformer block normalizes each position of each sequence in the batch by calculating the normalizing statistics across the channels. It is the complete opposite of batch normalization, in terms of how the normalization statistics are calculated. A diagram showing the difference between batch normalization and layer normalization is shown in Figure 9-7.

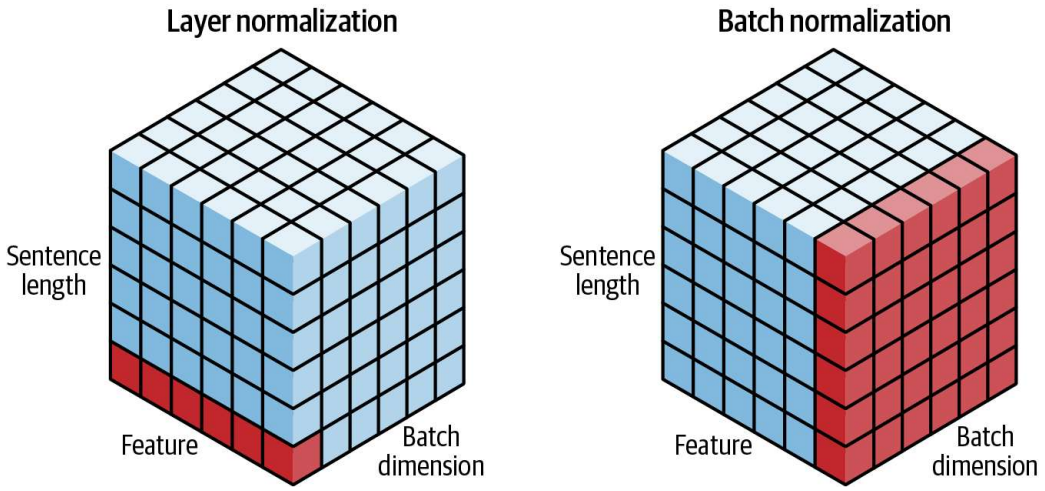

Figure 9-7. Layer normalization versus batch normalization—the normalization statistics are calculated across the blue cells (source: Sheng et al., 2020)[4]

Layer Normalization Versus Batch Normalization

Layer normalization was used in the original GPT paper and is commonly used for text-based tasks to avoid creating normalization dependencies across sequences in the batch. However, recent work such as Shen et al.*s* challenges this assumption, showing that with some tweaks a form of batch normalization can still be used within Transformers, outperforming more traditional layer normalization.

Lastly, a set of feed-forward (i.e., densely connected) layers is included in the Transformer block, to allow the component to extract higher-level features as we go deeper into the network.